

SYSTEM DESIGN INTERVIEW

Design Netflix

Video streaming platform serving 250M+ subscribers, 15M concurrent streams globally

~ 35 minute read

© Study Guide — For personal use only

What's Inside

- 01** · Problem Statement & Clarifying Questions
- 02** · Functional & Non-Functional Requirements
- 03** · Capacity Estimation
- 04** · High-Level Architecture
- 05** · Open Connect CDN Deep Dive
- 06** · Video Encoding Pipeline
- 07** · Adaptive Streaming (DASH)
- 08** · Microservices & Resilience
- 09** · Database Architecture
- 10** · Recommendation System
- 11** · Data Pipeline

- 12** · Resilience & Chaos Engineering
- 13** · Failure Modes & Recovery
- 14** · Trade-offs & Design Decisions
- 15** · Interview Playbook

01

Problem Statement & Clarifying Questions

Define the scope

Design **Netflix**: a global video streaming platform with 250M+ subscribers, 15M concurrent streams at peak, a catalog of ~5K titles, and personalized homepages per profile. The system must deliver high-quality video with sub-second startup, <0.5% stall rate, and DRM-protected playback across 190+ countries.

Clarifying Questions

Scope	Question	Assumption
Video Types	Streaming only, or include downloads?	Streaming primary; downloads secondary
Live Content	Live TV / sports, or VOD only?	Primarily VOD; live is edge case
DRM	Content protection required?	Yes — Widevine, PlayReady, FairPlay
Profiles	Multiple user profiles per account?	Yes — child / adult; recommendations per profile
Geographic	Global deployment required?	Yes — 190+ countries, localized CDN
Recommendations	Personalized homepage essential?	Yes — three-stage ranking pipeline
Offline	Offline viewing (downloads)?	Yes but secondary focus

We focus on the **streaming architecture for VOD content globally**, with DRM, personalized recommendations, and multi-profile support.

02

Functional & Non-Functional Requirements

What the system must do

Functional Requirements

- **Browse / search catalog** (metadata API): instant (<100ms)
- **Personalized recommendations**: list 100 tiles in <200ms
- **Initiate playback**: fetch OCA URL + DRM token within <500ms
- **Stream video continuously**: adaptive bitrate, seamless quality switches
- **Track viewing history**: record play / pause / seek / stop events
- **Multi-profile management**: switch profiles instantly
- **Download for offline**: queue + background sync

Non-Functional Requirements

Requirement	Target
Availability	99.95% uptime; graceful degradation if recommendation/billing fails
Latency	Initial playback <3s; segment delivery <50ms p99; UI <100ms
Scale	250M subscribers; 15M concurrent streams; 5K+ titles; 700B events/day
Quality	Adaptive bitrate (240p–4K); device codecs; multi-lang audio/subs
Security	End-to-end DRM; HTTPS; rate limiting; account security
Cost	Minimize CDN egress; aggressive caching; strategic content placement

03

Capacity Estimation

Math for scale

Users & Concurrency

- **250M subscribers**; 30–40% watch on any given day (~100M DAU)
- **Peak hour**: 15M concurrent streams (typical 8–10M; peak ~15M in US/EU primetime)
- **Device mix**: 40% TV, 45% mobile, 15% web/tablet
- **Avg session**: 1–2 hours; **avg bitrate 5 Mbps** (range 1–25 Mbps per device profile)

Bandwidth & CDN

- **Peak CDN traffic**: 15M streams × 5 Mbps = **75 Tbps**
- Annual content egress: **~350–400 EB** (exabytes) from origin/caches to users
- **OCA hit rate**: 99%+ from ISP-embedded appliances; <1% miss → regional PoP or origin
- Cache efficiency: proactive nightly push of top-N content; reactive caching for long-tail

Storage

- **Catalog size**: ~5K titles; ~100 GB per title (all variants) = **~500 TB aggregate**
- **Viewing history**: ~500B records (user_id, title_id, position, ts); ~100 GB hot in Cassandra
- **Metadata**: ~100 GB (posters, descriptions, reviews, ratings, availability matrix)
- **Event logs**: 700B events/day × 7-day retention ≈ **~2 PB** in Kafka / cloud

Cache Layer

- **EVCache (Memcached)**: ~800M queries/day (metadata, viewing history, OCA URL); 99% hit rate target; ~10 TB heap
- **Elasticsearch**: ~2 TB for searchable catalog + ratings/reviews; sharded by region

KEY

Key Numbers

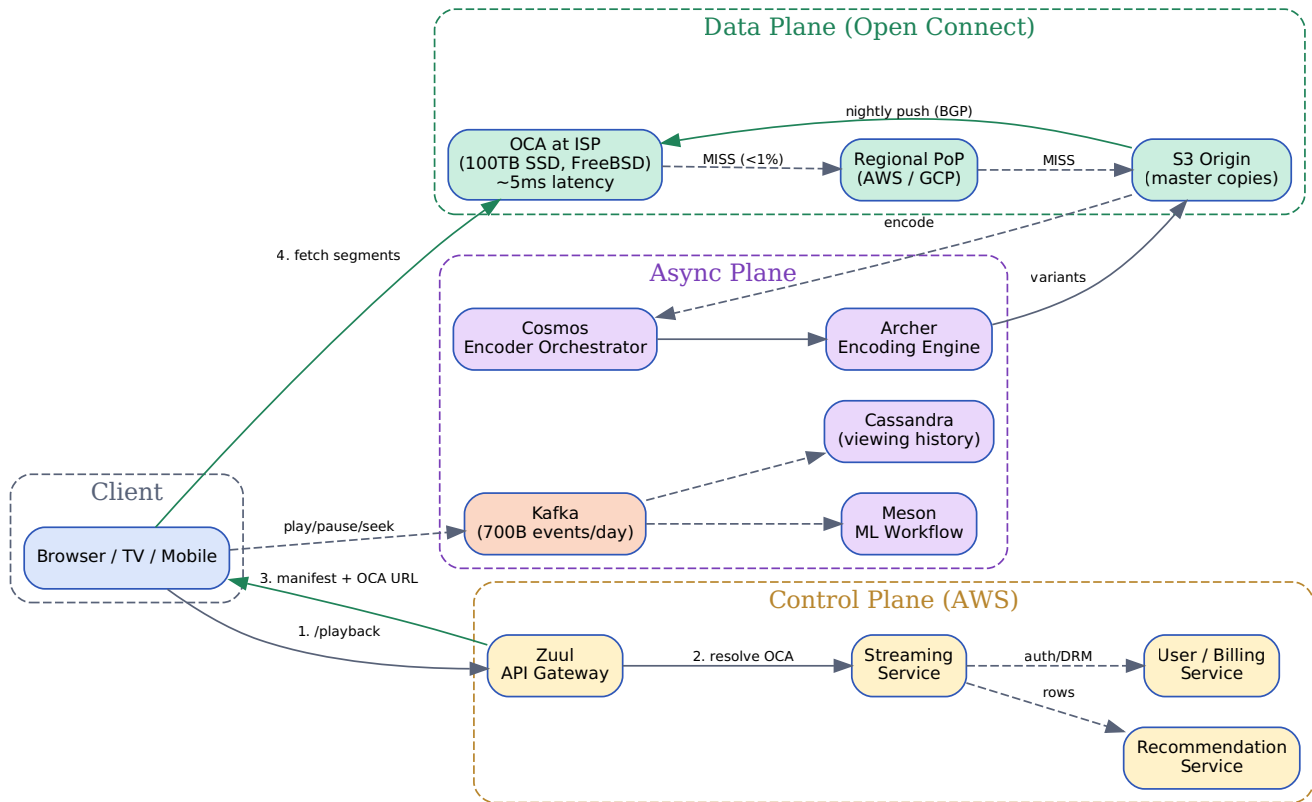
15M concurrent streams · **75 Tbps** peak CDN traffic · OCA hit rate **99%+** · ~500 TB catalog · 700B events/day · ~10 TB EVCache heap.

04

High-Level Architecture

Three planes — control, data, async

Netflix separates the system into three distinct planes: a **Control Plane** on AWS for orchestration, a **Data Plane** on the Open Connect CDN for video delivery, and an **Async Plane** for event processing and ML.



Playback flow: client resolves an OCA URL via the Control Plane, then streams segments directly from the ISP-embedded appliance, falling back to a regional PoP or S3 origin only on a cache miss.

Control Plane (AWS)

- **Zuul API Gateway:** request routing, auth, rate limiting; filters for logging, metrics, DRM tokens
- **User Service:** account, profiles, settings; backed by MySQL (ACID for billing)
- **Streaming Service:** core orchestrator; resolves OCA endpoint for client IP; queries EVCache for metadata / URL mappings
- **Recommendation Service:** three-stage ranking — candidate gen → filter → rank (150+ features)
- **Billing Service:** subscription state, payment processing, entitlements; MySQL + event journal

Data Plane (Open Connect CDN)

- **OCA Appliances:** custom hardware (100TB SSD, FreeBSD) embedded in 1,000+ ISPs globally; serves 99%+ of segments at ~5ms latency
- **Regional PoPs:** fallback caches in major cloud regions (AWS / GCP); serve cache misses from OCA
- **S3 Origin:** master video copies; proactive nightly push to OCAs via BGP-controlled distribution

Async Plane (Event-Driven)

- **Kafka:** ingest 700B events/day (play, pause, seek, rate, search, errors); partitioned by event type; 7-day retention
- **Spark:** daily batch jobs — model retraining, offline analytics, data warehouse ETL
- **Flink:** real-time stream processing — trending content, live dashboards, anomaly detection
- **Meson:** ML workflow orchestrator; chains feature gen, training, evaluation, deployment

05

Open Connect CDN Deep Dive

Why Netflix doesn't use AWS CloudFront

Netflix's proprietary CDN, **Open Connect (OCA)**, is the backbone of the data plane. **Cost** and **performance** are the drivers.

Why Custom CDN?

- **Cost:** CloudFront egress ~\$0.085/GB; OCA \$0.01–\$0.02/GB (5–10× cheaper via ISP partnerships, shared infra, bulk agreements)
- **Control:** content placement, eviction, codec/resolution variants per ISP; no third-party constraints
- **Scale:** 30–40% of global internet traffic is Netflix; no commercial CDN can compete on efficiency at that volume
- **Latency:** ISP-embedded appliances serve from local network — **~5ms vs 20–50ms** from regional edge

OCA Hardware & Deployment

- **Appliance specs:** custom **FreeBSD** appliance; **100 TB SSD**; 10 Gbps NIC; low-power, silent for ISP NOCs
- **Placement:** 1,000+ ISPs across 190+ countries; serves ~50–60% of peak global traffic
- **Regional PoPs:** AWS/GCP regions + backbone providers as fallback for OCA misses

Content Distribution Strategy

- **Proactive caching (nightly push):** Netflix predicts top-N content per ISP based on regional popularity, time zone, device profile; BGP-announced routes push during off-peak hours
- **Reactive caching:** OCA miss → fetch from regional PoP or S3; cache result for future requests (LRU eviction)
- **Cache hit rate:** 99%+ for top-50% of catalog; 90%+ for long-tail
- **Pre-position new releases:** trending content + new releases auto-sync nightly

Segment & Manifest Delivery

- **Manifests:** device-specific DASH manifests returned by Streaming Service; include optimal OCA URL for client's ISP
- **Segments:** client fetches 2–4 second MPEG-DASH segments directly from OCA; bitrate adapts per segment
- **Fallback logic:** if OCA unavailable, client retries regional PoP, then origin

INSIGHT

Why ISP-embedded matters

Embedding OCAs inside ISP networks turns Netflix's egress into the ISP's *internal* traffic — cheaper for the ISP (less peering bandwidth) and faster for the user (one network hop). The arrangement is mutually beneficial, which is why ISPs host the appliances for free.

06

Video Encoding Pipeline

Cosmos & Archer orchestration

Netflix encodes every title into **100+ variants** (codec / resolution / bitrate combinations) to optimise for device capability and network conditions.

Per-Title Encoding (Revolutionary)

- **Shot detection:** analyse master file for shot boundaries, scene complexity, motion intensity
- **Variable bitrate by scene:** complex scenes (explosions, CGI) get higher bitrate; simple scenes (dialogue, static) get lower
- **Result:** 20–30% bandwidth savings vs uniform bitrate at the same perceptual quality
- **Example:** 1080p H.264 averages 5 Mbps but rises to 8 Mbps for action and falls to 2 Mbps for dialogue

Codec Ladder & Device Profiles

Device Profile	Codec Preference	Resolutions	Bitrate Range
Smart TV (2023+)	HEVC (H.265), VP9	4K, 1080p	8–25 Mbps
Modern Smartphone	H.264, VP9	1080p, 720p	3–8 Mbps
Older Mobile / Tablet	H.264 only	720p, 480p	1–4 Mbps
Web Browser	H.264, VP9	1080p, 720p	3–8 Mbps
Fire Stick	H.264, HEVC	4K, 1080p	5–15 Mbps

Cosmos Orchestration

- **Cosmos:** distributed encoding orchestrator; schedules parallel transcoding across the encoding fleet
- **Archer:** encoding engine; runs FFmpeg (with custom plugins for per-title bitrate optimisation) on CPU/GPU clusters
- **Parallel encoding:** each title encoded to all variants in parallel (4K HEVC, 1080p H.264, 720p VP9, AAC audio, etc.)
- **Cost:** ~\$0.25–\$0.50 per hour of video encoded; 5K titles × 100 variants = ~500K encoding jobs; ~\$2–5M annual encoding spend

Packaging & Distribution

- **DASH** (Dynamic Adaptive Streaming over HTTP): industry-standard format; client-side adaptive bitrate selection
- **Segmentation**: 2–4 second segments; manifest lists all available qualities; player chooses based on bandwidth
- **DRM packaging**: Widevine, PlayReady, FairPlay licences embedded per variant
- **S3 storage**: master copies; nightly batch push to OCAs via BGP-directed distribution

07

Adaptive Streaming (DASH)

Client-side quality selection

Playback Flow

1. Client calls Zuul /playback with title_id, device profile, user IP
2. Streaming Service queries EVCache for OCA URL mapping (geo-aware; nearest ISP appliance)
3. EVCache returns optimal OCA endpoint for client's IP range
4. Zuul returns playback URL + DASH manifest to client
5. Client fetches DASH manifest from manifest server (metadata)
6. Client initiates segment fetch from OCA endpoint
7. OCA serves segment from local cache (HIT 99%+) or fetches from regional PoP (MISS <1%)

BOLA Algorithm (Buffer-Occupancy Bitrate Adaptation)

- **Goal:** maximise bitrate while avoiding stalls; minimise re-buffering events
- **Key metric:** buffer occupancy (seconds of video buffered locally), not network bandwidth alone
- **Logic:** if buffer > 30s, increase bitrate; if buffer < 10s, decrease quality; target ~15s buffer
- **Advantage:** more stable quality switching than network-speed-only; reduces stalls in congested networks
- **SLA:** stall rate < 0.5% (less than 3 seconds stalled per hour of viewing)

Segment Details

- **Segment duration:** 2–4 seconds; granular quality switching; ~50KB–5MB per segment depending on quality
- **Prefetching:** client prefetches 3–5 segments ahead to maintain buffer
- **Geo-aware URL:** IPv4 GeoIP lookup determines nearest OCA appliance
- **Fallback chain:** OCA → Regional PoP → S3 origin; client retries with exponential backoff

Stateless Services

- **Zuul, Streaming, Recommendation:** all stateless; horizontally scalable; any instance can serve any request
- **No local caches or sessions:** all state in EVCache or databases; failures don't lose in-flight state
- **Load balancing:** Eureka (service discovery) + Ribbon (client-side LB) for intelligent request routing

Circuit Breaker (Resilience Pattern — `resilience4j` / `concurrency-limits`)

WARNING

Hystrix is in maintenance mode

Netflix put **Hystrix into maintenance mode in 2018**; new services use **`resilience4j`** (or Spring Cloud Circuit Breaker), and Netflix internally adopted **adaptive concurrency limits** via the `concurrency-limits` library. The patterns below describe what those libraries implement; the library names matter when an interviewer probes.

- **Fallback catalog:** if Recommendation Service fails, fall back to 'Popular Now' or 'Trending Globally' (pre-computed offline)
- **Circuit-breaker logic:** fail fast; don't retry downstream if a service is down; return cached response or 'degraded' UI
- **Metrics:** error rate, latency p99, timeouts trigger OPEN; HALF_OPEN allows slow recovery testing
- **Timeouts:** all inter-service calls have <500ms timeout; fail fast rather than cascade latency
- **Adaptive concurrency:** the `concurrency-limits` library adjusts in-flight call limits using TCP-Vegas-style gradient feedback, shedding load before queues blow up

Chaos Engineering

- **Chaos Monkey:** randomly kills instances during business hours; teams must architect for recovery
- **Chaos Kong:** kills entire AWS region; tests multi-region failover (Route53 latency-based routing)
- **Latency Monkey:** injects artificial delays to test timeout handling and fallback logic
- **Culture:** failure injection is proactive testing; failures found in chaos tests don't surprise production

Service Discovery (Eureka) & Client-Side LB (Ribbon)

- **Eureka:** central service registry; services heartbeat every 30s; Netflix's internal DNS alternative

- **Ribbon:** client-side LB; caches instance list from Eureka; retries failed requests against healthy instances
- **Benefit:** no single LB bottleneck; routing considers instance health, latency zones, canary deployments

09

Database Architecture

MySQL + Cassandra + EVCache + Elasticsearch

MySQL (Billing, Users, ACID)

- **Use case:** user accounts, subscription tier, billing history, payment methods, entitlements
- **Why MySQL:** ACID essential; billing must be precise; transactions required for payment processing
- **Scale:** ~250M user records; write-heavy (daily billing, subscription changes); replicated across multi-master setup
- **Sharding:** by user_id modulo; thousands of shards to distribute load

Cassandra (Viewing History, Write-Heavy)

- **Use case:** viewing history (play position, watched status, resume timestamp); ~500B records; append-only log
- **Why Cassandra:** extremely write-heavy; distributed; no SPOF; tunable consistency (eventual ok for history)
- **Data model:** wide rows by user_id; columns = {title_id: {position, ts, watched}}
- **Throughput:** ~2.5K ops/sec per node; >100 nodes per cluster; millions of writes/sec aggregate
- **Consistency:** eventual (RF=3, quorum reads/writes for critical data)

EVCache (Memcached, Metadata & Playback URLs)

- **Use case:** metadata (title, poster, description), OCA playback URLs, viewing-history snippets
- **Scale:** ~800M hits/day; 99% hit rate target; <30ms p99 latency
- **Cluster:** ~10 TB total heap; distributed across hundreds of nodes; replicated 2–3×
- **Eviction:** LRU when memory full; keys expire after 24h (auto-refresh from source)
- **Key benefit:** avoids expensive Cassandra or S3 lookups; metadata in-memory

Elasticsearch (Search & Catalog)

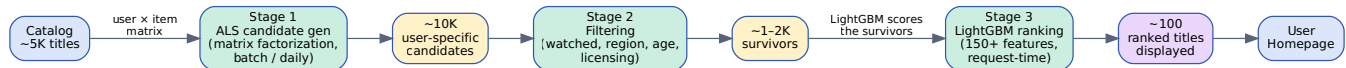
- **Use case:** full-text search (title, description, actor, director); faceted (genre, year, rating)
- **Scale:** ~2 TB index across 50+ shards; sharded by region/language for locality
- **Consistency:** near real-time; new titles indexed within minutes
- **Features:** typo tolerance, synonym matching, personalised results (rating history influences ranking)

10

Recommendation System

Three-stage ranking pipeline

Netflix's homepage personalises ~100 rows per user, each with 10–30 titles. The algorithm runs in three stages to balance quality and speed.



Recommendation funnel: a ~5K-title catalog narrows monotonically through ALS candidate generation, business-rule filtering, and LightGBM ranking before the top ~100 land in the user's feed.

Stage 1 — Candidate Generation (~10K candidates per user)

- **Algorithm:** Alternating Least Squares (ALS) matrix factorisation; collaborative filtering
- **Input:** user-title interaction matrix (viewing history, ratings, search, clicks); item embeddings (metadata features)
- **Output:** top ~10K candidate titles per user (not yet ranked; sorted by relevance score)
- **Scale:** batch job; retrains daily on Spark; maps users and titles to low-dimensional vectors
- **Latency:** generation is offline; runtime lookup is $O(1)$ hash or in-memory vector search

Stage 2 — Filtering (~1–2K candidates after filtering)

- **Rules:** remove already-watched, region-locked, or age-restricted titles for the profile
- **Business logic:** prioritise licensed content expiring soon; boost originals; remove low-quality titles
- **Latency:** <100ms; deterministic rules; computed in real time or cached

Stage 3 — Ranking (~100 titles to display; per-row ranking)

- **Algorithm:** Gradient Boosting (LightGBM); predicts CTR (click-through rate) for each title in context
- **Features (150+):** user (age, country, tier), item (genre, IMDb score, release date), context (time-of-day, device, row position)
- **Training:** offline daily batch; Spark pipeline on Kafka events (clicks, watches, searches)
- **Personalisation:** artwork selection per user (e.g., thumbnail of an actor the user has rated highly; or an action scene if the user watches action)
- **A/B testing:** homepage layout, row ordering, thumbnail variants — winner picked on CTR + watch time

Why Three Stages?

- **Speed:** each stage narrows the set monotonically — **~5K-title catalog** → **~10K user-specific candidates (Stage 1 ALS)** → **~1–2K after filtering (Stage 2)** → **~100 ranked titles displayed (Stage 3)**; LightGBM only scores the ~1–2K survivors, not the full catalog
- **Quality:** specialised algorithms at each stage; ALS captures broad taste; LightGBM captures interaction effects
- **Cost:** ALS + filtering offline; only LightGBM runs at request time; <200ms total latency
- **Explainability:** can inspect why a title ranked highly (similar title watched; high score; popular now)

INSIGHT

Why ALS expands before filtering shrinks

Stage 1 deliberately produces ~10K candidates — *more* than the 5K catalog size — because the ALS scoring is per-user and unique titles can appear with different relevance scores across multiple synthetic 'rows' (e.g. action, trending-in-country). Filtering then collapses duplicates and drops ineligible titles down to ~1–2K survivors before LightGBM does the expensive ranking.

11

Data Pipeline

Kafka → Spark / Flink → Analytics

Kafka Ingest (700B Events/Day)

- **Event types:** play, pause, seek, stop, rate, search, error, subscription change, profile switch
- **Volume:** 250M users × 2–3 events/min during watching = 700B events/day
- **Schema:** JSON; timestamp, user_id, title_id, device, playback position, event type, client IP
- **Retention:** 7 days hot; older events archived to S3 for long-term analytics
- **Partitioning:** by user_id or event_type; ensures ordering per user

Spark Batch Pipeline (Daily)

- **Jobs:** model retraining (ALS, LightGBM), data warehouse ETL, offline analytics, reporting
- **Input:** Kafka events + viewing history (Cassandra snapshots) + content metadata
- **Output:** trained models (ALS embeddings, LightGBM weights) → online serving; aggregated metrics → Druid
- **Latency:** ~4 hours per daily job; models deployed by morning for next day's recommendations

Flink Real-Time Stream (Live Metrics)

- **Jobs:** trending content (titles gaining play rate in last hour), anomaly detection (errors, stalls), live dashboards
- **Latency:** <5 second lag; results published to Kafka topics or websockets for dashboards
- **Stateful:** count plays per title per minute; detect $>3\sigma$ deviations as anomalies

Druid OLAP (Ad-Hoc Analytics)

- **Use case:** dashboard queries (titles watched per country, device, tier); near real-time exploration
- **Metrics:** plays, stall rate, bitrate distribution, recommendations CTR
- **Latency:** <1s for pre-aggregated dashboards; <5s for ad-hoc analysis

Multi-Region Active-Active Deployment

- **Regions:** US-East, US-West, EU-West, APAC (minimum 3 active regions)
- **Traffic routing:** Route53 latency-based routing → user lands on nearest region
- **Database replication:** MySQL multi-master (cross-region); Cassandra multi-DC (RF=3 across DCs)
- **Failover:** if a region fails, Route53 reroutes traffic automatically — no manual intervention
- **Data loss:** RPO <5 minutes (replication lag); RTO <30 seconds (Route53 + LB failover)

Chaos Monkey (Instance-Level Failures)

- **Action:** randomly terminates instances during business hours (production for some teams; staging for others)
- **Frequency:** approximately one instance per service per region per hour
- **Culture:** teams must be on-call; assume instances will fail; architect accordingly
- **Benefit:** uncovers single points of failure before they cause production incidents

Chaos Kong (Region-Level Failures)

- **Action:** simulates entire AWS region failure; yanks network cables (test mode in staging)
- **Test:** verify Route53 failover, DB failover, client reconnection logic all work
- **Cadence:** quarterly; part of disaster recovery drill

Latency Monkey (Network Delays)

- **Action:** injects 100–500ms artificial delay on inter-service calls
- **Purpose:** ensure all timeouts are configured correctly; fallback logic triggers before timeout
- **Result:** prevents cascade failures when one service degrades; circuit breaker opens gracefully

13

Failure Modes & Recovery

8 critical scenarios

Failure	Cause	Impact	Mitigation
OCA Appliance Outage	Hardware failure; ISP power loss	Users in that ISP routed to regional PoP; latency 5ms → 50ms; 1–5% QoE impact	Health checks; automatic fallback chain; redundant appliances per ISP
Cassandra Partition	Network split; DC unreachable	Viewing-history writes fail; reads serve stale data (eventual consistency)	Quorum reads/writes; RF=3 across regions; application fallback to cache
EVCache Miss Storm	Memcached cluster crash; cache stampede	Metadata lookups hit Cassandra; p99 latency spikes to 500ms	Circuit breaker; fallback to cached manifest; preload hot keys at startup
Recommendation Lag	Model training delayed; batch job failure	Homepage shows generic 'Popular Now' instead of personalised rows	Serve previous day's model; A/B impact minimal (CTR diff <1%)
Encoding Pipeline Failure	Cosmos orchestrator down; encoder crash	New title cannot be encoded; release delayed	Manual restart; on-demand encoding fallback; pre-compute common profiles
DRM Service Down	Licence server unavailable	Playback blocked; users cannot watch any content	Cache DRM licences locally for 24h; offline viewing if downloaded
Zuul Gateway Overload	Thundering herd; DDoS attack	API latency >1s; some requests time out and retry	Rate limiting; circuit breaker; shed load via 429 (client backoff)
Billing Service Failure	MySQL replication lag; payment processor down	Subscription state unknown; allow stream to continue (billing retries later)	Optimistic UI; async reconciliation; manual dispute resolution

WARNING

Graceful Degradation Hierarchy

Always preserve playback (the core function). Degrade in this order: personalised recommendations → custom artwork → search ranking quality → billing reconciliation. Never block a paid user from watching content they're entitled to because of a non-essential service failing.

14

Trade-offs & Design Decisions

Why these choices?

Decision	Alternative	Why Netflix Chose It
Own CDN (OCA)	AWS CloudFront / Akamai	Cost (5–10× cheaper); control over content placement; latency (ISP embedding)
Cassandra	DynamoDB / MongoDB	Write throughput (2.5K ops/sec/node); no SPOF; operational maturity
DASH (HTTP)	RTMP / custom protocol	Firewall-friendly; standards-based; client-controlled adaptive bitrate
Eventual Consistency	Strong consistency	Viewing history 'eventual ok'; availability + latency >> precision
Separate 3 Planes	Monolith	Independent scaling; failure isolation; team ownership
Three-Stage Ranking	Single model	<200ms latency achieved by filtering + LightGBM only on ~1–2K candidates
Daily Model Retraining	Real-time updates	Cost / complexity trade-off; daily refresh sufficient for CTR; A/B catches bugs
Proactive CDN Push	Reactive only	Bandwidth savings; 99%+ hit rate; scale advantages at Netflix's size
resilience4j / concurrency-limits	Hystrix	Hystrix in maintenance since 2018; resilience4j is actively developed; concurrency-limits handles adaptive shedding

15

Interview Playbook

How to present this design (45 minutes)

Opening (5 min): Clarifying Questions & Scope

- Start with 2–3 clarifying questions: VOD only? Global? Recommendations essential?
- Confirm assumptions: 250M subscribers, 15M concurrent streams, ~5K titles, multi-region
- Set scope: focus on streaming architecture (not downloads or live)

Capacity (2 min)

- Memorise: 250M subscribers, 15M concurrent (peak), **75 Tbps** CDN traffic, ~500 TB catalog
- Draw on whiteboard: user base → concurrent → bandwidth calculation
- Point out the scale problem: standard CDN (CloudFront) would be cost-prohibitive

Architecture (10 min): 3 Planes + Diagrams

- **Control plane:** Zuul → microservices (User, Billing, Streaming, Recommendation); stateless + circuit breakers
- **Data plane:** Open Connect CDN; 1,000+ ISP-embedded appliances; 99%+ hit rate; nightly proactive push
- **Async plane:** Kafka (700B events/day) → Spark (daily retraining) → Flink (live metrics)
- Use diagrams; explain why custom CDN is critical for cost & scale

Deep Dives (20 min): Pick 2–3 Topics

- **CDN:** OCA hardware, ISP partnerships, proactive vs reactive caching; why 99%+ hit rate
- **Recommendations:** three-stage ranking; **5K** → **10K** → **1–2K** → **100**; 150+ features; personalisation
- **Encoding:** per-title encoding; scene-based variable bitrate; codec ladder; Cosmos orchestration
- **Resilience:** Chaos Monkey/Kong; resilience4j circuit breaker; multi-region active-active
- Common probes: 'How would you handle a Cassandra partition?', 'Why not use CloudFront?', 'How do you scale recommendations?'

Wrap-Up (5 min): Trade-offs

- Discuss cost vs quality vs latency trade-offs
- Mention operational complexity (Chaos Monkey culture, multi-region databases)
- End with: *Netflix is unique at scale; most companies don't justify a custom CDN, but Netflix streams 30–40% of global internet traffic*

Key Numbers to Memorise

- **250M subscribers, 15M concurrent streams, 75 Tbps** CDN traffic
- **OCA hit rate 99%+**; latency 5ms vs 50ms regional
- **700B events/day** to Kafka; daily model retraining
- Cassandra: 2.5K ops/sec/node; EVCache: 99% hit rate; 800M queries/day
- DASH 2–4s segments; BOLA buffer algorithm; <0.5% stall-rate SLA
- Encoding: 100+ variants per title; per-title VBR saves 20–30% bandwidth
- Recommendation funnel: **~5K → ~10K → ~1–2K → ~100**

What Interviewers Are Testing

- **Systems thinking:** can you connect pieces? (Why custom CDN affects encoding and caching strategy)
- **Scale intuition:** 15M concurrent = how much bandwidth? How many DB ops?
- **Trade-off reasoning:** why OCA over CloudFront? Why Cassandra over MySQL?
- **Resilience:** what fails? How do you recover? Do you have a fallback?
- **Communication:** can you explain a complex system simply? Use diagrams. Admit uncertainty.

KEY

Memorise These

75 Tbps peak · 99%+ OCA hit · 700B events/day · 5K → 10K → 1–2K → 100 funnel · resilience4j (not Hystrix) · 5ms vs 50ms latency.